

TRUY VẤN DỮ LIỆU

Bài 5. Học xếp hạng

PGS.TS. Lê Thanh Hương
Trường Công nghệ thông tin và Truyền thông
Đại học Bách Khoa Hà Nội

Các mô hình xếp hạng

- Đối với một mô hình cụ thể
 - Khó để chọn tham số tối ưu, đặc biệt là khi có nhiều tham số
- Để so sánh giữa hai mô hình
 - Với một bộ thử nghiệm, rất khó để so sánh hai mô hình, khi mỗi hình được điều chỉnh quá khớp trong khi mô hình kia thì không
- Đối với một tập các mô hình
 - Khó để kết hợp chúng một cách hiệu quả
- Học máy có thể:
 - Tự động tinh chỉnh các thông số của mô hình
 - Tránh các vấn đề quá khớp

Cách xếp hạng truyền thống

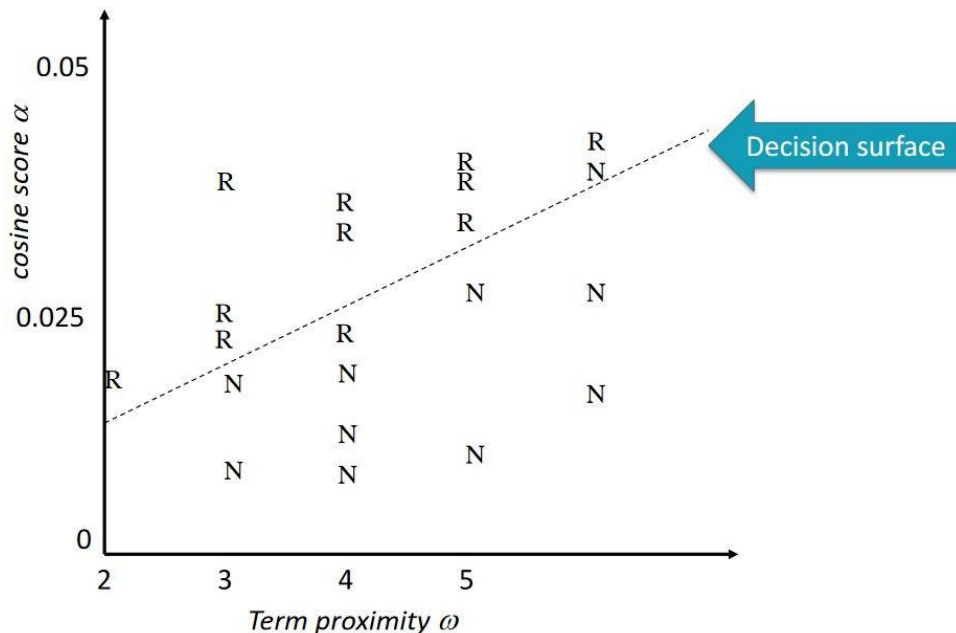
- Các hàm xếp hạng truyền thống trong IR sử dụng ít đặc trưng, ví dụ:
 - Term frequency
 - Inverse document frequency
 - Document length
- Có thể dễ dàng điều chỉnh các hệ số trọng số bằng tay

Cách xếp hạng dùng học máy

- Các hệ thống hiện đại sử dụng nhiều đặc trưng:
 - Các đặc trưng tùy thuộc ứng dụng
 - Tần suất của từ truy vấn trong anchor text?
 - Từ truy vấn từ màu trên trang?
 - Số hình ảnh trên trang?
 - Số liên kết trên trang?
 - PageRank của trang?
 - Độ dài URL?
 - URL chứa "~"?
 - Lần cuối chỉnh sửa trang?
 - Tốc độ tải trang

Coi xếp hạng như bài toán phân loại

- Thu thập tập huấn luyện với các mẫu (q, d, r)
 - Độ liên quan r là giá trị nhị phân (thực tế có thể là đa lớp)



Learning to Rank

- Giả sử độ liên quan được sắp theo các mức

$$c_1 < c_2 < \dots < c_J$$

- Giả sử tập huấn luyện bao gồm các cặp tài liệu-truy vấn, ở dạng các vector đặc trưng ψ và thứ hạng liên quan c_i .

Learning to Rank

- **Input:** tập mẫu (là các đặc trưng của **query và documents**)

$$X = \{x_1, x_2, \dots, x_n\}$$

- **Output:** danh sách xếp hạng các mẫu dựa trên hàm score

$$Y' = \{x_{r1}, x_{r2}, \dots, x_{rn}\}$$

- **Mục tiêu:** tối ưu việc xếp hạng dựa trên các ma trận NDCG, MAP, MRR (đã học ở bài 3)

$$Y = \{x_{y1}, x_{y2}, \dots, x_{yn}\}$$

Nhắc lại

- MAP - Mean Average Precision

$$MAP = \frac{1}{|Q|} \cdot \sum \left(\frac{1}{R_q} \cdot \sum P@K_i \right)$$

- NDCG - Normalized Discounted Cumulative Gain.

$$DCG = rel_1 + rel_2/\log_2 2 + \dots rel_n/\log_2 n$$

- MRR - **M**ean **R**eciprocal **R**ank (Trung bình của giá trị nghịch đảo vị trí của kết quả phù hợp đầu tiên)

$$MRR(Q) = \frac{1}{|Q|} \cdot \sum_{q \in Q} \frac{1}{K_q}$$

Training & Test Data

Training

Relevance	Doc ID	Query ID	Query Length	Doc Length	Title SIM	Body SIM
1	1023	q1	2	103	0.50	0.45
2	27	q1	2	20	0.75	0.69
0	1023	q2	5	103	0.03	0.05
3	1532	q2	5	89	0.90	0.88
0	5	q3	1	1077	0.10	0.06

Not Seen in Training

Query Feature

Document Feature

Query-Document Feature

Test

Relevance	Doc ID	Query ID	Query Length	Doc Length	Title SIM	Body SIM
?	104	q1	2	105	0.80	0.72

Predict



Phân loại mô hình học xếp hạng

- **Pointwise:** gán nhãn mức độ liên quan cho 1 tài liệu
 - Đầu vào: 1 tài liệu
 - Đầu ra: điểm hoặc nhãn mức độ liên quan
- **Pairwise:** Xác định thứ hạng của 2 tài liệu
 - Đầu vào: cặp tài liệu
 - Đầu ra: thứ tự ưu tiên trong cặp
- **Listwise:** xác định thứ tự cả tập tài liệu
 - Đầu vào: tập hợp tài liệu
 - Đầu ra: danh sách tài liệu được xếp hạng

Phân loại mô hình học xếp hạng

Given a query q and a set of documents $D = (d_1, \dots, d_n)$:

Pointwise

Input: Single candidate
 $x = (q, d_i)$



Loss: How accurate is the predicted score $s_i \approx y_i$?



Solution: Transform task into Regression.

Pairwise

Input: Pair of candidates
 $x_i = (q, d_i)$ and $x_j = (q, d_j)$



Loss: If $y_i > y_j$, then are the predicted scores $s_i > s_j$?



Solution: Transform task into Binary Classification.

Listwise

Input: Whole list of candidates
 $x_1 = (q, d_1) \dots x_n = (q, d_n)$



Loss: Takes into account position of all retrieved docs.



Solution: Incorporate evaluation metrics (e.g. DCG) into loss.

From [Francesco Casalegno](#) Post on towarddatascience



Pointwise Approach

Minimize the squared loss
between the estimated
relevance score and the
true relevance score

perfect, excellent, good,
fair, and bad



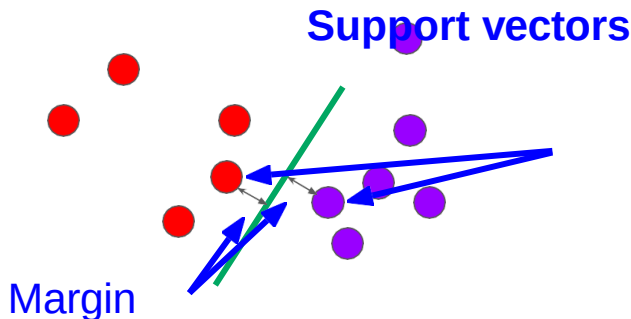
	Regression	Classification	Ordinal Regression
Input	Input vector of features: x		
Output	Real Number $y = f(x)$	Category $y = \text{sign}(f(x))$	Ordered Category $y = \text{thresh}(f(x))$
Model	Ranking Function $f(x)$		
Loss	Regression Loss	Classification Loss	Ordinal Regression Loss

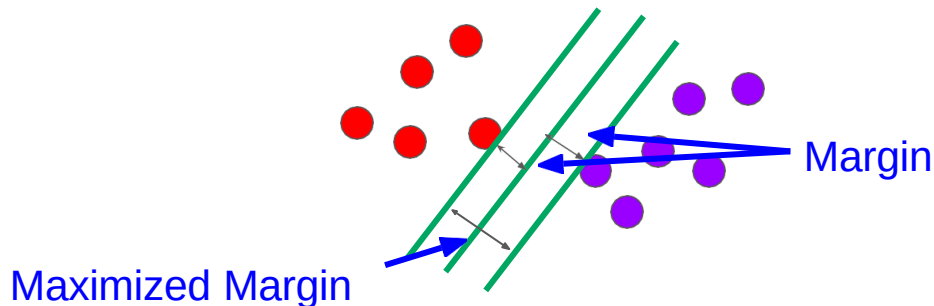
Pairwise Approach

- Với một cặp tài liệu, phương pháp này học nhằm đưa ra thứ tự tối ưu cho cặp đó và so sánh với thứ tự chuẩn.
- Mục tiêu:
 - Giảm số lần đảo ngược trong quá trình xếp hạng.
 - Giảm số trường hợp mà cặp kết quả bị xếp sai thứ tự so với thứ tự chuẩn.

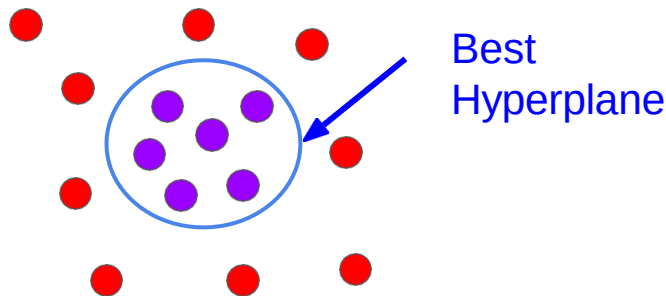
SVMRank - Ranking SVM (Thorsten Joachims, 2002)

- Dựa trên Support Vector Machines (SVM), giải quyết các bài toán xếp hạng trong IR. Nó áp dụng nguyên lý tối ưu hóa của SVM để xây dựng một mô hình học xếp hạng dựa trên việc so sánh các cặp tài liệu và tối ưu hóa thứ hạng của chúng.
- SVM là một mô hình tuyến tính dùng cho phân loại, tạo ra một đường thẳng hoặc siêu phẳng để phân chia dữ liệu thành các lớp.
- **Support vectors:** các điểm dữ liệu gần nhất với siêu phẳng (các yếu tố quan trọng của tập dữ liệu).
Margin: Khoảng cách giữa siêu phẳng và điểm dữ liệu gần nhất từ mỗi tập.





- Mặt siêu phẳng nào tốt hơn?
- SVM cố gắng tạo ra một biên sao cho khoảng cách giữa hai lớp là rộng nhất có thể.



- Đôi khi cần thêm chiều mới để tìm được siêu phẳng tốt nhất
- SVM thực tế không cần tìm ra các vector này. SVM có thể hoạt động chỉ với các tích vô hướng giữa chúng (Kernel function).

- SVMRank xếp hạng tài liệu dựa trên các cặp tài liệu và tối ưu hóa thứ tự của chúng thông qua kỹ thuật phân loại SVM
- **Đầu vào:**
 - Mỗi truy vấn q có một danh sách các tài liệu liên quan $d_1, d_2, \dots, d_n$ với nhãn (label) cho biết mức độ liên quan của từng tài liệu.
- **Mục tiêu:** xếp hạng các tài liệu sao cho tài liệu có nhãn cao hơn (liên quan hơn) sẽ đứng trước các tài liệu có nhãn thấp hơn.
- **Tạo ra các cặp tài liệu:**
 - Từ danh sách các tài liệu được gán nhãn, ta tạo ra các cặp tài liệu.

Xây dựng mô hình:

- SVMRank học 1 hàm tuyến tính dựa trên các cặp tài liệu (d_i, d_j) , với mục tiêu tối ưu hóa một hyperplane nhằm phân tách tốt nhất các (d_i, d_j) , nghĩa là tài liệu có liên quan hơn sẽ nằm ở phía đúng của siêu phẳng so với tài liệu kém liên quan hơn.
- Ràng buộc với hàm mục tiêu

$$w \cdot (x_i - x_j) \geq 1 - \xi_{ij}, \forall (d_i, d_j)$$

- x_i và x_j là các đặc trưng của tài liệu d_i và d_j .
- w là vector trọng số được học bởi mô hình.
- ξ_{ij} là biến sai số cho phép.

- **Mục tiêu:** tối thiểu hóa hàm mất mát dựa trên lỗi xếp hạng và chuẩn của vector trọng số w :

$$\min \frac{1}{2} ||w||^2 + C \sum \xi_{ij}$$

- C là tham số điều chỉnh sự đánh đổi giữa độ phức tạp của mô hình và lỗi huấn luyện.
- **Xếp hạng tài liệu:**
 - Sau khi huấn luyện, điểm cho các tài liệu được tính thông qua hàm $f(x)=w \cdot x$ với x là vector đặc trưng của tài liệu.

Ưu điểm:

1. **Hiệu quả trong việc xếp hạng:** SVMRank có thể đưa ra thứ hạng chính xác bằng cách tối ưu hóa thứ tự giữa các cặp tài liệu
2. **Áp dụng dễ dàng với các mô hình IR**

Nhược điểm:

1. **Tốn tài nguyên tính toán**
2. **Khả năng mở rộng:** SVMRank có thể gặp khó khăn khi xử lý các tập dữ liệu lớn do độ phức tạp tính toán cao khi làm việc với nhiều cặp tài liệu.

Pairwise Approaches

- **Ưu điểm**

- Tiến gần hơn một bước đến việc xếp hạng tài liệu so với việc dự đoán nhãn phân loại.

- **Nhược điểm**

- Một số truy vấn có nhiều cặp truy vấn hơn so với các truy vấn khác.
- Vẫn chưa tối ưu cho các độ đo trong IR
- Không nhận biết thứ hạng — ($d_1 > d_2$) không thể hiện rõ thứ hạng nào đang được so sánh: Hạng 1 với Hạng 2 hay Hạng 99 với Hạng 1000.

- **RankNet** (Microsoft Research), sử dụng mạng nơ-ron để xếp hạng tài liệu dựa trên mức độ liên quan
- RankNet học cách so sánh và xếp hạng hai tài liệu
- Sử dụng thuật toán gradient descent để tối ưu hóa hàm mất mát cross-entropy, nhằm giảm thiểu số lượng các vị trí xếp hạng sai

Burges, Christopher JC, Robert Ragno, and Quoc Viet Le. "Learning to rank with nonsmooth cost functions." NIPS. Vol. 6. 2006 Burges, Christopher JC. From ranknet to lambdarank to lambdamart: An overview. Learning, 11(23-581), 2010

Listwise Approach

- Các mô hình học được thiết kế để tối ưu chất lượng toàn bộ danh sách kết quả xếp hạng, thay vì chỉ tối ưu cho từng cặp tài liệu (pairwise) hoặc từng tài liệu riêng lẻ (pointwise).
- Tập trung vào việc tối ưu các chỉ số xếp hạng như NDCG, MAP, MRR
- Các phương pháp tiêu biểu: **AdaRank**, **LambdaRank**, **LambdaMART**

- AdaRank là một thuật toán **boosting** để xếp hạng danh sách. Thay vì chỉ học một mô hình duy nhất, AdaRank kết hợp nhiều mô hình xếp hạng yếu (weak rankers) để tạo thành một mô hình mạnh hơn.
- **Hoạt động:**
 - AdaRank tìm cách tối ưu trực tiếp các chỉ số đánh giá danh sách, ví dụ như NDCG.
 - Thực hiện quá trình lặp lại để huấn luyện từng mô hình con (weak ranker), và tại mỗi bước lặp, nó gán trọng số cao hơn cho những truy vấn mà mô hình hiện tại không xếp hạng tốt. Sau mỗi lần học, mô hình sẽ cập nhật lại trọng số cho các truy vấn, từ đó cải thiện độ chính xác trên các truy vấn khó.

- **Ưu điểm:**

- Tối ưu trực tiếp các chỉ số xếp hạng.
- Có thể đạt hiệu quả cao khi kết hợp các mô hình yếu thành một mô hình mạnh.

- **Nhược điểm:**

- Dễ overfitting nếu không có cơ chế dừng hợp lý.
- Không có khả năng mở rộng tốt cho dữ liệu lớn do cần huấn luyện nhiều mô hình con.

- Trọng tâm của LambdaRank là tối ưu các chỉ số xếp hạng (như NDCG)
- Hoạt động:
 - Dựa trên việc điều chỉnh "lambda gradients" - các gradient được tính toán không chỉ dựa trên sự khác biệt trong điểm số giữa các cặp tài liệu, mà còn dựa trên sự thay đổi trong các chỉ số xếp hạng như NDCG.
 - Khi thay đổi vị trí của hai tài liệu trong danh sách, sự thay đổi về NDCG sẽ được sử dụng để điều chỉnh giá trị gradient trong quá trình huấn luyện.

- **Ưu điểm:**

- Tối ưu trực tiếp cho các chỉ số xếp hạng quan trọng như NDCG, giúp cải thiện hiệu quả xếp hạng tổng thể.
- Sử dụng gradient để điều chỉnh xếp hạng một cách linh hoạt.

- **Nhược điểm:**

- LambdaRank yêu cầu tính toán gradient dựa trên toàn bộ danh sách, có thể gây ra vấn đề về tốc độ khi làm việc với các tập dữ liệu lớn.

LambdaMART

- LambdaMART là một phiên bản mở rộng của LambdaRank, nhưng sử dụng gradient boosted decision trees (GBDT) thay vì mạng neural trong quá trình học xếp hạng. Nó kết hợp những ưu điểm của LambdaRank (tối ưu hóa trực tiếp các chỉ số xếp hạng) và GBDT (hiệu quả, dễ mở rộng).
- Hoạt động:
 - LambdaMART sử dụng cây quyết định tăng cường (boosted decision trees) để dự đoán điểm số của các tài liệu.
 - Tương tự LambdaRank, nó cũng điều chỉnh giá trị gradient dựa trên sự thay đổi của các chỉ số xếp hạng như NDCG khi thay đổi vị trí của tài liệu trong danh sách.
 - LambdaMART huấn luyện một chuỗi các cây quyết định, mỗi cây sau sẽ cố gắng sửa lỗi của các cây trước đó.

• Ưu điểm:

- Tận dụng được sức mạnh của GBDT, cho phép huấn luyện nhanh hơn và có khả năng mở rộng tốt hơn so với các mô hình neural trong LambdaRank.
- Hiệu quả tốt trong việc tối ưu hóa các chỉ số xếp hạng phức tạp như NDCG.
- Được sử dụng rộng rãi và đã chứng minh hiệu quả trong thực tiễn.

• Nhược điểm:

- Mặc dù có khả năng mở rộng tốt hơn so với LambdaRank, nhưng với các tập dữ liệu cực kỳ lớn, LambdaMART vẫn có thể gặp vấn đề về hiệu suất.

Đánh giá cách tiếp cận Listwise

- **Ưu điểm**

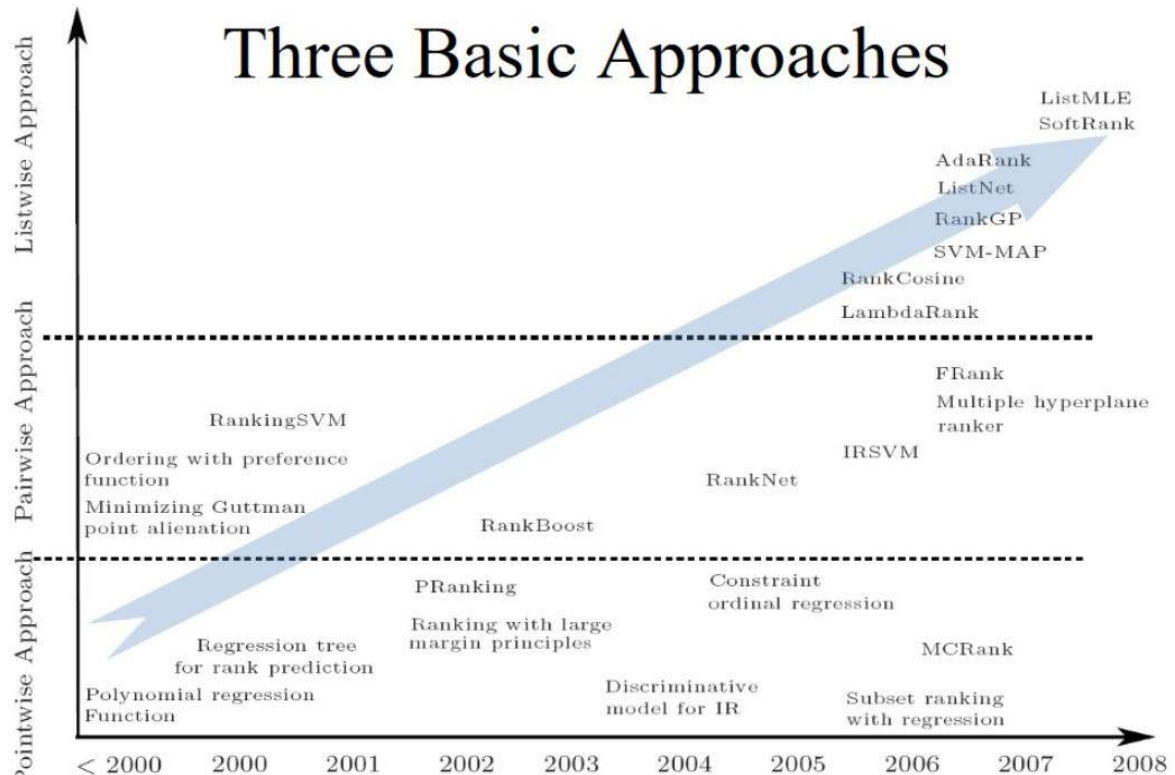
- Sử dụng tất cả các tài liệu liên quan đến một truy vấn làm ví dụ học
- Hàm loss có xét đến vị trí xếp hạng

- **Nhược điểm:**

- Cần nhiều dữ liệu huấn luyện

Cách tiếp cận tốt nhất

- listwise ranking > pairwise > pointwise



Học không giám sát

- **CombSUM:** Kết hợp kết quả xếp hạng từ nhiều hệ thống tìm kiếm hoặc nhiều truy vấn khác nhau.
- Thuật toán này là một trong các phương pháp hợp nhất điểm số) nhằm cải thiện chất lượng và độ chính xác của kết quả truy hồi thông tin.
- Giả sử có N hệ thống IR khác nhau trả về điểm số cho tài liệu d . Điểm số của tài liệu d từ hệ thống thứ i là $score_i(d)$.

$$Score_{\text{CombSUM}}(d) = \sum_{i=1}^N score_i(d)$$

- Tài liệu có điểm số cao hơn sẽ đứng đầu danh sách kết quả.

Giả sử có 3 hệ thống truy hồi thông tin khác nhau. Mỗi hệ thống trả về điểm số cho tài liệu d:

- Hệ thống 1: $\text{score}_1(d)=0.7$
- Hệ thống 2: $\text{score}_2(d)=0.8$
- Hệ thống 3: $\text{score}_3(d)=0.9$

Tổng điểm của tài liệu d:

$$\text{Score}_{\text{CombSUM}}(d) = 0.7 + 0.8 + 0.9 = 2.4$$

Đặc điểm của CombSUM

Ưu điểm:

- Đơn giản: chỉ cần cộng các điểm số từ nhiều hệ thống.
- Tính tổng quát: không cần điều chỉnh các hệ thống IR ban đầu, mà chỉ cần lấy tổng điểm số
- Hiệu quả trong nhiều trường hợp: CombSUM có thể mang lại kết quả tốt hơn khi các hệ thống IR khác phương pháp hoặc khác bộ dữ liệu.

Hạn chế:

- Không xem xét độ tin cậy của từng hệ thống
- Không chuẩn hóa điểm số

- Điểm số của một tài liệu d là tổng các điểm số nhân với số hệ thống thực sự trả về tài liệu đó

$$Score_{\text{CombMNZ}}(d) = Count(d) \times \sum_{i=1}^N score_i(d)$$

Count(d) là số lượng hệ thống trả về tài liệu d

- Mặc dù có những vấn đề về chuẩn hóa thường gặp trong các phương pháp dựa trên điểm số, CombMNZ vẫn cạnh tranh với các phương pháp dựa trên xếp hạng

Reciprocal Rank Fusion (RRF)

- RRF hoạt động bằng cách tổng hợp thứ hạng của các tài liệu từ nhiều nguồn khác nhau và đưa ra một danh sách xếp hạng kết hợp dựa trên công thức **reciprocal rank**
- RRF giả định nếu một tài liệu xuất hiện trong nhiều danh sách xếp hạng và có thứ hạng cao trong bất kỳ danh sách nào, thì tài liệu đó có khả năng liên quan cao hơn.
- **Hàm RRF** gán trọng số cho mỗi tài liệu theo nghịch đảo của vị trí của nó trong bảng xếp hạng
 - Ưu tiên các tài liệu ở "đầu" bảng xếp hạng
 - Phạt các tài liệu ở dưới "đầu" bảng xếp hạng

Reciprocal Rank Fusion (RRF)

- Giả sử có K danh sách xếp hạng từ các hệ thống hoặc mô hình khác nhau, với mỗi danh sách bao gồm các tài liệu $d_1, d_2, \dots, d_n$. Tài liệu d có thứ hạng $r_k(d)$ trong danh sách thứ k
- Tính điểm cho mỗi tài liệu:** Điểm của tài liệu d là tổng các nghịch đảo của thứ hạng mà nó nhận được trong mỗi danh sách

$$S(d) = \sum_{k=1}^K \frac{1}{r_k(d) + c}$$

- $r_k(d)$ là thứ hạng của tài liệu d trong danh sách thứ k .
- c là một hằng số nhỏ (thường được đặt là 60) nhằm tránh các tài liệu có thứ hạng quá cao (~ 1) chiếm ưu thế quá lớn.
- Xếp hạng tài liệu dựa trên điểm:** Tài liệu nào có điểm cao hơn sẽ được xếp hạng cao hơn trong danh sách.

Reciprocal Rank Fusion (RRF)

Ví dụ: Giả sử có hai mô hình M1 và M2 với các tài liệu được xếp hạng như sau:

- M1 xếp hạng: d1 (hạng 1), d2 (hạng 2), d3 (hạng 3).
- M2 xếp hạng: d2 (hạng 1), d3 (hạng 2), d1 (hạng 3).
- Với $c=60$, điểm số của mỗi tài liệu:

$$S(d_1) = \frac{1}{1+60} + \frac{1}{3+60} = \frac{1}{61} + \frac{1}{63}$$

$$S(d_2) = \frac{1}{2+60} + \frac{1}{1+60} = \frac{1}{62} + \frac{1}{61}$$

$$S(d_3) = \frac{1}{3+60} + \frac{1}{2+60} = \frac{1}{63} + \frac{1}{62}$$

- Xếp hạng lại dựa trên điểm số đã tính để có danh sách xếp hạng kết hợp.

Đặc điểm của RRF

- **Kết hợp thông tin từ nhiều nguồn:** tận dụng ưu điểm của mỗi mô hình.
- **Đơn giản nhưng hiệu quả:** Mặc dù RRF chỉ dựa vào thứ hạng (rank), nó thường đạt hiệu quả cao trong IR
- **Ít nhạy cảm với nhiễu:** Bằng cách sử dụng nghịch đảo thứ hạng, RRF ít bị ảnh hưởng bởi các tài liệu có thứ hạng kém trong một mô hình nhưng lại được xếp hạng cao trong các mô hình khác.

Ưu điểm:

1. **Khả năng tổng hợp linh hoạt**
2. **Hiệu quả cao trên nhiều bài toán**
3. **Không yêu cầu điều chỉnh tham số phức tạp:** Ngoại trừ hằng số c , RRF không đòi hỏi việc điều chỉnh quá nhiều tham số

Nhược điểm :

1. **Phụ thuộc vào số lượng mô hình:** phụ thuộc vào chất lượng và sự đa dạng của các mô hình được kết hợp
2. **Thiên về thứ hạng hơn là điểm:** có thể làm mất đi một số thông tin chi tiết mà các mô hình đã học được.

Unsupervised Reranking Methods

- ranx là một thư viện xếp hạng nhanh các chỉ số đánh giá trên Python, tận dụng Numba để thực hiện các phép toán vector tốc độ cao và tự động song song hóa

The link for ranx GitHub:

<https://github.com/AmenRa/ranx>



The link for the notebooks:

<https://github.com/AmenRa/ranx/tree/master/notebooks>

Installing ranx command:

```
! pip install ranx
```